A beam search for the air cargo palletization problem with weight distribution constraints

Ignacio Araya
Escuela de Ingeniería Informática, Pontificia Universidad Católica de Valparaíso,
Avenida Brasil 2950, Valparaíso, Chile
ignacio.araya@pucv.cl

October 23, 2026

Abstract

We address the palletization of air cargo into unit load devices (ULDs): a set of heterogeneous boxes must be loaded into as few ULDs as possible, where the ULDs have bevelled corners to fit the fuselage, a weight capacity and a floor edge for the cargo net locks, the boxes have allowed orientations and may be fragile, every box must be supported, and the center of gravity of each ULD must lie in a given area around its geometric center. We adapt a block-based beam search guided by a greedy algorithm (BSG), originally designed for the container loading problem, to this problem. The blocks are generated and placed so that the support and fragility constraints are met, blocks that cross a bevelled side are shifted to the nearest feasible position, and the center of gravity is taken into account in the evaluation of the blocks and of the solutions, with a final translation of the load, which is kept grouped by anchoring the free spaces at one corner of the ULD. The ULDs are loaded sequentially, and the solution is improved by eliminating ULDs and by inserting the unloaded boxes with ruin and recreate. On the 300 benchmark instances of Paquay et al. and with the same parameters in all the settings, the algorithm uses 8–16% fewer ULDs than the state of the art when the number of ULDs is unlimited, being worse in at most one instance; with one ULD of each type, it leaves fewer boxes unloaded with fewer ULDs. Almost none of its ULDs violates the center of gravity constraint, against 7–11% of the ULDs of the previous method.

Keywords: packing; air transport; three-dimensional bin packing; beam search; weight distribution

1 Introduction

Before a cargo aircraft takes off, the individual shipments are consolidated onto air cargo pallets and into containers, the so-called unit load devices (ULDs), which are then loaded into the aircraft. The shape of a ULD depends on its position in the aircraft: lower-deck containers have bevelled bottom corners and main-deck pallets and containers have bevelled top corners, so that they fit the fuselage. Building the ULDs is a daily operational task in air freight terminals, where it has to be solved for many flights in a short time, and every ULD saved is space and weight that can be sold or saved in fuel. At the same time, a ULD plan is only usable if it respects the practical constraints of air transport: the weight capacity of each ULD, the allowed orientations of the boxes, fragile boxes on which nothing can be placed, the stability of the load, and the weight distribution, since the center of gravity (CoG) of each ULD must lie in a given area around its geometric center (Paquay et al.,

2016). Building the ULDs is one of the subproblems of the air cargo load planning problem (Brandt and Nickel, 2019).

Assuming that the cargo consists of rectangular boxes, the problem is a three-dimensional Multiple Bin Size Bin Packing Problem (MBSBPP) in the typology of Wäscher et al. (2007), with non-cuboid bins and additional constraints. Paquay et al. (2016) proposed a mixed integer programming formulation of the problem, which can only solve small instances, and a set of instances derived from a real set of 562 boxes and the six ULD types of a Boeing 777. Paquay et al. (2018b) and Paquay et al. (2018a) proposed MIP-based and two-phase constructive heuristics for these instances. Recently, Heßler et al. (2025) extended the problem with further practical features (a floor edge for the cargo net locks, on which no box can be loaded, a substructure to raise the floor, and padding material to support the boxes) and proposed a randomized greedy search (RGS) based on a fast extreme-point insertion heuristic. The RGS uses 30% fewer ULDs than the heuristic of Paquay et al. (2018a) in a fraction of a second per instance, and it is, to the best of our knowledge, the state of the art for these instances.

In this paper we show that a different algorithmic paradigm, borrowed from the single container loading problem, yields considerably better ULD plans. The beam search guided by a greedy (BSG) algorithm (Araya and Riff, 2014; Araya et al., 2017) loads a container with *blocks* of identical or similar boxes placed in *maximal free spaces*, and selects the blocks with a beam search in which each node is evaluated by completing its partial solution with a greedy algorithm. It is among the best algorithms for the classical container loading problem (Araya et al., 2017, 2020), but the air cargo problem differs from it in several respects: the ULDs are not cuboids, the support of the boxes and the fragile boxes restrict how boxes can be stacked, the CoG of every ULD is constrained, several ULDs of different types must be loaded, and all the boxes must be loaded with as few ULDs as possible. Our contributions are the following:

- We adapt BSG to load a ULD with bevelled corners, a floor edge and an optional substructure. The blocks are generated so that every box inside a block is supported, and each block is placed in a free space only if the boxes of its base are supported according to the corner or area criterion of the problem and no box touches a fragile box. A block that crosses a bevelled side of the ULD is shifted along the length of the ULD to the nearest feasible position.

- We handle the CoG constraint in three ways: the evaluation of the blocks is penalized by the CoG deviation that the placement would cause, the value of a solution is penalized by the excess of the CoG deviation over the allowed one, and the final load of each ULD is translated as a whole to center its CoG. To leave room for this translation, all the free spaces are anchored at the corner closest to the origin of the ULD, which keeps the load grouped.

- We embed the single-ULD algorithm into a sequential multi-ULD framework with a post-optimization phase that tries to eliminate ULDs by redistributing their boxes into the other ULDs and, when the number of ULDs is limited, a ruin-and-recreate phase that inserts the boxes left unloaded into the loaded ULDs.

- We tune the parameters once with irace on a separate training set and evaluate the algorithm on the 300 instances of Paquay et al. (2018a) in the three settings considered by Heßler et al. (2025). With the same parameters in all the settings, our algorithm uses 8–16% fewer ULDs than the RGS in the settings with an unlimited number of ULDs, where it is better in up to 111 of the 300 instances and worse in at most one; with one ULD of each type, it leaves fewer boxes unloaded and uses 3–7% fewer ULDs. Almost none of its ULDs violates the CoG constraint, against 7–11% of the ULDs of the RGS. All the solutions are checked by an independent validator, and the code, the solutions and the scripts are publicly available.

The remainder of the paper is organized as follows. Section 2 defines the problem and Section 3 reviews the related literature. Section 4 describes the algorithm, Section 5 reports the computational experiments, and Section 6 concludes the paper.

2 Problem definition

We follow the problem definition of Paquay et al. (2016) with the extensions of Heßler et al. (2025). The length, width and height of a ULD correspond to the x, y and z axes, respectively; z is the vertical axis.

Boxes. A set I of rectangular boxes must be loaded. Each box $i \in I$ has a length l_i, a width w_i, a height h_i and a weight q_i. The boxes are placed orthogonally, and each box has a set of allowed orientations: for each of its three dimensions, the instance states whether the box can be placed with that dimension in the vertical direction (each allowed vertical dimension gives two orientations, rotated or not around the vertical axis). A box can be fragile, in which case nothing can be placed on top of it. The weight of a box is uniformly distributed, so that its CoG is its geometric center. The instance files of Paquay et al. (2018a) also flag very dense boxes; as Heßler et al. (2025), we do not use this flag.

ULDs. A set T of ULD types is available, in unlimited number or with a given number n_k of ULDs of each type $k \in T$. A ULD of type k is the bounding box $[0, L_k] \times [0, W_k] \times [0, H_k]$ from which up to four corners are cut by planes orthogonal to the x-z plane, to fit the fuselage of the aircraft. We represent each cut by a half-plane $\{(x, z) : a\,x + b\,z \leq c\}$ in the x-z plane that contains the usable space; a box is inside the ULD if it is inside its bounding box and its projection on the x-z plane satisfies all the cuts. The ULD type also defines its usable volume V_k, its weight capacity Q_k, the maximum deviations d_k^x and d_k^y of the CoG of the load from the geometric center in the length and width directions, and the maximum height d_k^z of the CoG. Following Heßler et al. (2025), the floor of the ULD may have an edge of width e for the cargo net locks: no box can be loaded over the edge below the height δ (the vertical edge offset). To load large boxes at the bottom, a substructure of height δ can be used, which raises the floor (without the edge) to the height δ; in that case no box can be loaded below the height δ.

Constraints. A loading plan assigns each loaded box to a ULD, an orientation and a position. It is feasible if:

1. each box is inside its ULD and, if the ULD has an edge, it does not overlap the edge below the height δ;
2. the boxes do not overlap;
3. the total weight of the boxes in each ULD of type k does not exceed Q_k;
4. each box whose base is not on the floor is supported: either the four corners of its base rest on other boxes (or on the floor or the substructure), or part of its base rests directly on other boxes and at least a proportion o of its base area rests directly on boxes or on padding material (Heßler et al., 2025); padding material can fill gaps of height at most $\hbar$ between the base of a box and the top of the boxes below;
5. no box, and no padding material, is placed on top of a fragile box;

6. the CoG $(\bar{x}, \bar{y}, \bar{z})$ of the load of each ULD satisfies $|\bar{x} - L_k/2| \leq d_k^x$, $|\bar{y} - W_k/2| \leq d_k^y$ and $\bar{z} \leq d_k^z$.

The original problem of Paquay et al. (2016) corresponds to $e = \delta = \hbar = 0$ and $o = 1$, that is, a box is supported only if the four corners of its base are supported. Paquay et al. (2018a) also allow the corners to rest on a bevelled wall of the ULD; we do not use this possibility, so our solutions are feasible for their definition too. Heßler et al. (2025) do not consider the vertical CoG constraint; we consider it in all the experiments. Moreover, both Paquay et al. (2018a) and Heßler et al. (2025) treat the horizontal CoG constraint as a soft constraint, since their solutions contain ULDs with the CoG out of range; we report the number of such ULDs for all the methods.

Objective. When the number of ULDs is unlimited, all the boxes must be loaded and the objective is to minimize the number of ULDs. When it is limited, some boxes may be left unloaded, and the objective is first to maximize the number of loaded boxes and then to minimize the number of ULDs. These are the criteria used in the instance-by-instance comparison of Section 5, and the ones reported by Heßler et al. (2025). Paquay et al. (2016) formulate the objective as the minimization of the unused volume, that is, of the total volume of the used ULDs, and Heßler et al. (2025) maximize the loaded volume and then minimize the total volume of the ULDs. Our algorithm minimizes the number of ULDs, and at the end it tries to move the boxes of its least utilized ULD into a smaller one (Section 4.4). Section 5.2 also reports the total volume and cost of the ULDs.

3 Related work

Three-dimensional packing problems with practical constraints have received much attention; we refer to Bortfeldt and Wäscher (2013) for a review of the constraints considered in container loading, to Zhao et al. (2016) for a comparison of container loading algorithms, and to Brandt and Nickel (2019) for a definition of the air cargo load planning problem and a review of its subproblems. We focus on the works closest to ours: air cargo palletization, packing with weight distribution constraints, and the heuristics on which our algorithm is based.

Air cargo palletization. Chan et al. (2006) developed a decision support system for loading air cargo pallets in a real case study. Paquay et al. (2016) formulated the problem considered in this paper as a mixed integer program, with non-cuboid ULDs, weight capacities, orientations, fragile boxes, support and CoG constraints, and solved small instances with a commercial solver. To solve larger instances, Paquay et al. (2018b) proposed three MIP-based constructive heuristics (relax-and-fix, insert-and-fix and fractional relax-and-fix), and Paquay et al. (2018a) a tailored two-phase constructive heuristic: the first phase packs the boxes into identical bins with an extension of the extreme points, and the second one deals with the several ULD types; if necessary, the weight distribution of the best loading is then improved in a post-processing step with shift and jump operators that move or repack boxes. Heßler et al. (2025) extended the problem with the ULD edge, the substructure and the padding material, and proposed a randomized greedy search (RGS). Each ULD is loaded by repeatedly calling an insertion heuristic with several sorting criteria of the boxes and a random perturbation of the order; the insertion heuristic places each box at the first feasible extreme point (Crainic et al., 2008), uses an extended set of extreme points that can be moved to fill the space under the bevelled sides of the ULD, and accelerates the feasibility checks with a grid of cells. The best loading of a ULD is selected with a score that combines the volume utilization and the weight balance, and the ULDs are loaded sequentially, first those that are needed for the boxes that fit in the fewest ULD types. Their algorithm improves the results of Paquay et al. (2018a) by

about 30% in a fraction of a second per instance. A related but different problem is the loading of the ULDs into the aircraft, which determines the position of each ULD to balance the aircraft, sometimes with several pickup and delivery points (Lurkin and Schyns, 2015; Mesquita and Sanches, 2026).

Weight distribution. Constraints on the CoG of the load arise in container loading (Davies and Bischoff, 1999), in pallet loading and in air cargo (Chan et al., 2006; Paquay et al., 2016), and they have also been studied in the three-dimensional knapsack problem (Baldi et al., 2012) and in the load-balanced bin packing problem (Trivella and Pisinger, 2016). Many heuristics handle them in a post-processing step, for instance by reordering and mirroring the walls of the load after the packing has been built (Davies and Bischoff, 1999) or by shifting and repacking boxes (Paquay et al., 2018a), or with a score that trades the weight balance against the volume utilization (Heßler et al., 2025). Paquay et al. (2018a) also consider the CoG during the construction, with a merit function that selects the extreme point that keeps the CoG closest to the center of the ULD. In our algorithm the CoG is taken into account in the evaluation of every candidate block and in the value of the partial solutions that guides the beam search, and the load is also translated at the end.

Heuristics for three-dimensional packing. Most heuristics for three-dimensional bin packing build the packing by placing one box at a time at a candidate position: the corner points of Martello et al. (2000), the extreme points of Crainic et al. (2008), or the maximal free spaces of Parreño et al. (2008). Parreño et al. (2008) and Parreño et al. (2010) embedded a maximal-space constructive heuristic in GRASP and variable neighbourhood descent, and Alvarez-Valdes et al. (2013) extended it with path relinking to the multiple bin size bin packing problem. Gajda et al. (2022) proposed a randomized constructive heuristic for a real container loading problem with weight distribution along the truck axles. For the single container loading problem, the best results have been obtained by tree search algorithms that place *blocks* of boxes instead of single boxes: Fanslau and Bortfeldt (2010) combined blocks of boxes with a partition-controlled tree search, Zhu and Lim (2012) proposed an iterative-doubling greedy lookahead algorithm, and Araya and Riff (2014) a beam search in which each node is evaluated by a greedy completion of its partial solution (BSG). Araya et al. (2017) introduced the VCS function to evaluate the blocks, which combines the volume of the block, its contact surface with the placed boxes and the container, the volume that the block makes unusable in the free space, and the number of boxes, Araya et al. (2020) extended BSG to a biobjective container loading problem that also maximizes the profit of the loaded boxes, and Guesser et al. (2026) improved its implementation with cache-friendly data structures, the memoization of the greedy completions and a two-stage evaluation function, obtaining the best results to date on the classical instances. Heßler et al. (2025) also tested the RGS on the container loading instances of Bischoff and Ratcliff (1995) and reported that it lags behind BSG with VCS (Araya et al., 2017), which they attribute to the much larger number of boxes per container in those instances. To the best of our knowledge, block-based tree search has not been applied to air cargo palletization, where the boxes are highly heterogeneous, the ULDs are not cuboids, and the support, fragility and CoG constraints restrict the blocks that can be built and where they can be placed. The main purpose of this paper is to show how to handle these features and that, with them, block-based tree search also outperforms the state of the art on this problem.

Finally, the elimination of ULDs and the insertion of the unloaded boxes in our multi-ULD framework are related to the local search heuristics that try to empty one bin at a time (Faroe et al., 2003) and to the ruin and recreate principle (Schrimpf et al., 2000).

4 Algorithm

The algorithm loads the ULDs one at a time. Each ULD is loaded by BSG, a beam search guided by a greedy algorithm, which builds the load with blocks of boxes placed in maximal free spaces (Section 4.1). The ULD-specific features (the bevelled corners, the edge and the substructure, and the support and fragility constraints) are handled in the generation and the placement of the blocks (Section 4.2), and the CoG constraint at several levels of the search (Section 4.3). Section 4.4 describes the multi-ULD framework.

4.1 Loading one ULD with BSG

Blocks. A block is a cuboid arrangement of boxes that is placed as a unit. Before loading a ULD, a set of blocks is generated from the boxes still available, as in Fanslau and Bortfeldt (2010) and Araya and Riff (2014): the blocks of one box are the boxes in their allowed orientations, and new blocks are obtained by joining two existing blocks along the x, y or z axis, so that every block has a guillotine structure. A joined block is kept only if its boxes are available, it fits in the bounding box of the ULD, the boxes fill at least a proportion $f_{\min}$ of its volume, and it is different from the blocks already generated (same dimensions and same boxes); the generation stops when $N_{\max}$ blocks have been generated ($N_{\max}$ is tuned, Section 5.1). We use $f_{\min} = 0.98$, so that the blocks are almost solid. Each block stores its weight and its CoG, computed recursively from the two blocks that form it, which allows us to evaluate the CoG of a load without visiting its boxes.

Free spaces and actions. The free space of the ULD is represented by the set of maximal free spaces, the largest cuboids that do not intersect the placed blocks (Parreño et al., 2008). When a block is placed, the free spaces that intersect it are replaced by their maximal parts outside the block, and the spaces too small for any remaining box are discarded. In Araya and Riff (2014), each space is anchored at its corner closest to a corner of the container, and the next space to fill is the one whose anchor corner is closest (in Manhattan distance) to the corresponding corner of the container, so that the load grows from all the corners towards the center. Here, instead, every space is anchored at its corner closest to the origin of the ULD and the next space is the one closest to the origin, so that the load grows from one corner and stays grouped: a grouped load leaves room to translate it, which is needed to center its CoG (Section 4.3), and when it does not fill the ULD it is not spread against several walls. Ties are broken by the larger volume. An action places a block that fits in the selected space at its anchor corner, which may be modified as explained in Section 4.2. A state of the search is a partial loading of the ULD, with its free spaces and the remaining boxes.

Evaluation of the blocks. The blocks that fit in the selected space are evaluated with the VCS function of Araya et al. (2017):

$$\mathrm{VCS}(b) = V(b)\,\big(1 - \mathrm{loss}(b)\big)^{\beta}\,\mathrm{CS}_p(b)^{\alpha}\,n(b)^{-\gamma}, \tag{1}$$

where $V(b)$ is the volume of the boxes of the block, $\mathrm{loss}(b)$ is the proportion of the volume of the free space that cannot be filled with the remaining boxes after placing the block (estimated with one-dimensional knapsack problems on each dimension of the space), $\mathrm{CS}_p(b)$ is the proportion of the surface of the block in contact with the placed blocks and the walls of the ULD, where faces at a distance of at most a proportion p of the block dimension are considered in contact, and $n(b)$ is the number of boxes of the block. The evaluation is penalized by the CoG deviation caused by the block, as explained in Section 4.3.

Greedy and beam search. The greedy algorithm repeatedly selects the next free space and places the best evaluated block in it, until no block can be placed. The value of a complete loading is its volume utilization penalized by the CoG excess (Section 4.3). BSG (Araya and Riff, 2014) is a beam search in which the children of each node are generated with the best evaluated blocks for the selected space, and each child is evaluated by the value of its greedy completion. At each level, the w children with the best greedy values are kept, and a total of at most m children are generated, so that each node of the beam is expanded with about m/w blocks. The best complete solution found by any greedy completion is returned. BSG is run iteratively with increasing effort until the time limit t is reached (Araya et al., 2017): it starts with $w = 4$ and $m = 16$, and after each run w and m/w are multiplied by $\sqrt{2}$. The implementation follows the optimizations proposed by Guesser et al. (2026) for BSG: the free spaces and the placed blocks are stored in contiguous arrays, the contact surface is computed only with the blocks near the selected space, the duplicate blocks are detected with a hash during the generation, and the greedy completions are memoized: the states visited by the complete greedy runs are stored (identified by a hash of their placed blocks), and when a greedy run reaches a known state, the rest of the solution is rebuilt from memory instead of evaluating the blocks again. In addition, the volume loss of a block is computed only when an upper bound of its evaluation can enter the best ones. None of these optimizations changes the result of the search.

4.2 ULD constraints

Bevelled corners. A block placed at the anchor corner of a space may cross one of the cuts of the ULD. In that case, the block is shifted along the x axis, inside the space, to the closest position that satisfies all the cuts, which can be computed in closed form from the half-planes of the cuts; if there is no such position, the block is not placed in the space. Since the cuts are orthogonal to the x-z plane, shifting the block along x keeps it on the same base, which is necessary for its support. The free spaces are still cuboids, so a space next to a cut may contain positions that are not usable; they are simply not used by any block.

Edge and substructure. With a floor edge of width e and offset δ, the initial free spaces are the space above the floor inside the edge and the space above the height δ over the whole floor. With a substructure, the only initial space is the one above δ, and the floor inside the edge is raised to δ. A space above the edge has no support until boxes have been loaded inside it; instead of discarding it, it is kept aside (*dormant*) and reconsidered after the next block is placed. Each ULD is loaded twice, with and without the substructure, and the best of the two loadings is kept.

Support and fragility. The support criterion of the problem (the four corners of the base, or the area criterion of Section 2) is checked box by box. Inside a block it is guaranteed by construction: two blocks are joined along z only if the lower block has a complete top face without fragile boxes and the upper block fits on it, so that every box of the upper block has its whole base, and in particular its four corners, on non-fragile boxes. Therefore, only the boxes at the bottom of a block have to be checked when the block is placed. A block is placed on the base of the selected space, and it is feasible if each box at its bottom is supported according to the criterion of the instance (the four corners of its base or the area criterion of Section 2), considering the top faces of the placed boxes at the height of its base (direct support) or at most $\hbar$ below it (support by padding material), and if it does not rest directly or through padding on a fragile box. Since the corner criterion allows gaps under a box, we also forbid placing a fragile box in such a gap if it would touch the base of the

box above. If no block can be placed in a space because of the support, the space is discarded or, with an edge, kept dormant.

4.3 Center of gravity

The CoG constraint is taken into account in three ways, besides the anchoring of the spaces at the origin.

Translation of the load. Once a ULD has been loaded, the whole load can be translated horizontally without changing its feasibility, as long as every block stays inside the ULD and its cuts and, if the ULD has an edge, the blocks below the height δ stay inside the edge and the blocks on the substructure stay on it. These conditions define an interval of feasible translations in x and in y, which can be updated incrementally when a block is placed. The final load of each ULD is translated by the feasible amount that brings its CoG closest to the center of the ULD.

CoG excess. Let $(\bar{x}, \bar{y}, \bar{z})$ be the CoG of a loading after the best feasible translation $(\Delta x, \Delta y)$. Its CoG excess is

$$\varepsilon = \frac{\left(|\bar{x} + \Delta x - L/2| - d^x\right)^+}{d^x} + \frac{\left(|\bar{y} + \Delta y - W/2| - d^y\right)^+}{d^y} + \frac{(\bar{z} - d^z)^+}{d^z}, \tag{2}$$

where $(a)^+ = \max(a, 0)$. The CoG constraint is satisfied if and only if $\varepsilon = 0$.

Value of a solution. The value of a (partial or complete) loading with volume utilization u is

$$v = \frac{u}{1 + \lambda\,\varepsilon}, \tag{3}$$

where $\lambda \geq 0$ is a parameter. This value is used to compare the greedy completions in BSG and to select the best loading of a ULD. A multiplicative penalty keeps the value of any non-empty loading positive, so that an unbalanced loading is still preferred to an empty ULD. With a large λ, the CoG constraint becomes almost a hard constraint; with $\lambda = 0$, it is ignored.

Evaluation of the blocks. To guide the construction towards balanced loads, the VCS evaluation (1) of a block is divided by $1 + \mu\,\varepsilon'$, where ε' is the CoG excess of the partial loading after placing the block, including the best feasible translation, and $\mu \geq 0$ is a parameter. The weight and the moments of the partial loading and its interval of feasible translations are stored in each state, so that ε' is computed in constant time for each candidate block.

Algorithm 1 summarizes how a ULD is loaded.

Algorithm 1 LoadULD(k, B): load a ULD of type k with the available boxes B.

```
S* ← empty loading
for s ∈ {without substructure, with substructure} do        ▷ only if the ULD has an edge
    generate the blocks for B and a ULD of type k with s
    S ← BSG(k, s, B, t)
    translate S to center its CoG
    if v(S) > v(S*) then S* ← S
return S*
```

4.4 Loading several ULDs

Algorithm 2 outlines the multi-ULD framework. It has a sequential construction phase followed by two improvement phases, which share the same insertion procedure, and a final replacement of the least utilized ULD by a smaller one.

Sequential construction. The ULDs are loaded one at a time with LoadULD, and the type of the next ULD is selected as in Heßler et al. (2025): among the ULD types into which the boxes that fit in the fewest available types fit, the type with the largest total volume of the remaining boxes that fit in it is selected (ties are broken by the larger ULD volume). A box fits in a ULD type if its weight does not exceed the capacity and, in some allowed orientation, it can be placed alone on the floor inside the edge or on the substructure. When the number of ULDs is limited, some boxes may only fit in the selected type among the ULDs still available. If the loading of the ULD leaves some of them out, the ULD is loaded again with these boxes first, and the load is then completed with the other boxes, keeping the first ones at their positions.

Insertion of boxes into the loaded ULDs. Both improvement phases rely on the same procedure, Insert, which inserts a pool of boxes into the ULDs already loaded, without adding ULDs. It applies three stages, each of which visits the ULDs in decreasing order of free volume; the new loading of a ULD is accepted if it loads more volume and its CoG is not out of range when the old one was not, and the boxes that the ULD no longer loads return to the pool:

1. *Reload from scratch*: the ULD is loaded again with LoadULD with its boxes and the boxes of the pool.
2. *Reload with priority*: the ULD is first loaded only with the boxes of the pool that fit in its type, and the load is then completed with its own boxes and the rest of the pool. This stage gives a chance to large boxes that do not fit in the free space left by the current load.
3. *Ruin and recreate* (Schrimpf et al., 2000): for $\rho \in \{0.3, 0.6, 0.9\}$, a proportion ρ of the loaded volume of each ULD is removed, taking the boxes with nothing on top of them from the highest one down, so that the remaining load is still feasible; then the ULDs are completed with the removed boxes and the pool, keeping the remaining boxes at their positions. The new solution is accepted if the volume of the pool decreases, and the stage is repeated while it decreases.

The procedure stops as soon as the pool is empty.

Unloaded boxes and elimination of ULDs. When the number of ULDs is limited and some boxes remain unloaded after the construction, Insert is applied to them. When all the boxes are loaded, the algorithm tries to eliminate ULDs: the ULDs are considered in increasing order of utilization, the boxes of the selected ULD are freed and inserted into the other ULDs with Insert, and if all of them are inserted, the ULD is eliminated and the phase starts again; otherwise the next ULD is tried. The phase ends when no ULD can be eliminated.

Smaller ULD. Finally, similarly to Heßler et al. (2025), who reload the last ULD, the boxes of the least utilized ULD are loaded into a smaller ULD type in which all of them fit, trying the types by increasing volume, and the change is accepted if all the boxes are loaded and the CoG is not out of range when it was not before.

Algorithm 2 Multi-ULD framework.

$B \leftarrow I$; $\mathcal{S} \leftarrow \emptyset$ ▷ remaining boxes, loaded ULDs
while $B \neq \emptyset$ **and** some available ULD type fits a box of B **do**
  $k \leftarrow$ next ULD type for B (Heßler et al., 2025)
  $S \leftarrow \textsc{LoadULD}(k, B)$
  if the ULDs are limited and S leaves out boxes that only fit in type k **then**
    load these boxes first and complete S with the other boxes of B
  $\mathcal{S} \leftarrow \mathcal{S} \cup \{S\}$; remove the boxes of S from B
if $B \neq \emptyset$ **then**
  $\textsc{Insert}(\mathcal{S}, B)$ ▷ unloaded boxes (limited ULDs)
else
  repeat ▷ elimination of ULDs
    for $S \in \mathcal{S}$ in increasing order of utilization **do**
      $B' \leftarrow$ boxes of S; $\textsc{Insert}(\mathcal{S} \setminus \{S\}, B')$
      if $B' = \emptyset$ **then** remove S from $\mathcal{S}$; **break**
  until no ULD was eliminated
try to move the boxes of the least utilized ULD into a smaller ULD
return $\mathcal{S}$, B

5 Computational experiments

This section evaluates the proposed algorithm on the benchmark of Paquay et al. (2018a) and on the two adapted variants introduced by Heßler et al. (2025), and compares it instance by instance with the randomized greedy search (RGS) of Heßler et al. (2025), which, to the best of our knowledge, reports the best results on these instances. Section 5.1 describes the instances, the implementation, the parameter tuning and the evaluation protocol. Section 5.2 reports the main results, Section 5.3 compares them with a lower bound, Section 5.4 analyzes the running time, Section 5.5 studies the contribution of each component and Section 5.6 the trade-off between the number of ULDs and the weight distribution.

5.1 Experimental setup

Instances. We use the 300 final instances of Paquay et al. (2018a): ten groups of 30 instances with $|I| \in \{10, 20, \ldots, 100\}$ boxes sampled from a real set of 562 boxes, highly heterogeneous, with weights, allowed vertical orientations and fragile (non-stackable) boxes. Six ULD types of a Boeing 777 are available in unlimited number: three lower-deck containers (types 1–3, with up to two bottom corners cut to fit the fuselage) and three main-deck pallets and containers (types 4–6, with up to two top corners cut). Each ULD type has a weight capacity and a maximum deviation of the center of gravity (CoG) from its geometric center in the length and width directions, and a maximum height of the CoG (Paquay et al., 2016). Following Heßler et al. (2025), we consider three settings:

- **Paquay**: the original instances. A box is supported if its four bottom corners rest on non-fragile boxes or on the floor, there is no padding material, and the allowed CoG deviations are those of the instance file.
- **Adapted**: the ULD floor has an edge of width $e = 10\,\text{mm}$ for the cargo net locks, on which no box can be loaded below the height $\delta = 10\,\text{mm}$; a substructure of height δ can be used to raise the floor. A box is supported if its four bottom corners are supported or if part of its base is

directly supported and at least 90% of its base is supported directly or by padding material of height at most 10 mm. The allowed CoG deviation is 10% of the half length and half width of the ULD. These are the default problem parameters of Heßler et al. (2025, Table 2).

- **One**: the adapted instances with only one ULD of each type, so that some boxes may remain unloaded. Solutions are compared lexicographically: first the number of loaded boxes, then the number of ULDs.

In the first two settings all the boxes must be loaded and the objective is to minimize the number of ULDs.

Implementation. The algorithm is implemented in C++ (compiled with g++ 13.3, `-O3`). Each run uses a single thread; the experiments were carried out on a server with two Intel Xeon E5-2620 v4 processors (8 cores each, 2.1 GHz, 3.0 GHz in turbo mode, hyper-threading disabled) and 62 GB of RAM, running 14 instances in parallel. According to the single-thread rating of PassMark (PassMark Software), one core of the Apple M2 Pro used by Heßler et al. (2025) is about 2.5 times faster than one core of our processor (4105 against 1624). The time limit t is the time of each run of BSG, excluding the generation of the blocks; we report results for $t = 0.1\,\text{s}$ and $t = 1\,\text{s}$. Since a ULD can be loaded several times (with and without the substructure and in the improvement phases), the total time per instance is several times larger, and we report it separately. The code, the instances, the solutions and the scripts that produce all the tables are available at `https://github.com/rilianx/bsg-uld`.

Parameter tuning. The parameters of the algorithm were tuned once with irace (López-Ibáñez et al., 2016), as Paquay et al. (2018a) did for their heuristic, on a separate training set and are used unchanged in the three settings and for both time limits. The training set consists of 150 preliminary instances of Paquay et al. (2018a) (30 instances with 30, 40, 50, 60 and 100 boxes), disjoint from the final ones, in the Paquay setting with $t = 0.1\,\text{s}$ and a budget of 3000 runs. The cost of a run is the number of ULDs plus five times the number of ULDs whose CoG is out of range, plus $(1-u)/2$, where u is the total utilization, to break ties in favour of smaller ULDs. Table 1 lists the tuned parameters.

Table 1: Parameters tuned with irace (training instances, Paquay setting, $t = 0.1\,\text{s}$).

Parameter	Description	Range	Value
λ (`cog_penalty`)	penalty of the CoG excess in the value of a solution	$[1, 50]$	12.66
μ (`cog_greedy`)	penalty of the CoG excess in the evaluation of a block	$[0, 5]$	0.599
$N_{\max}$ (`max_bl`)	maximum number of blocks	$\{500, 1000, 2000, 4000\}$	500
α	exponent of the contact surface (VCS)	$[0, 5]$	1.234
β	exponent of the volume loss (VCS)	$[0, 5]$	0.239
γ	exponent of the number of boxes (VCS)	$[0, 1]$	0.224
p	tolerance of the pseudo contact surface (VCS)	$[0, 0.1]$	0.057

Validation and comparison. Every solution is written to a file (position and orientation of each box) and checked by an independent validator, which does not share code with the solver: each box is loaded at most once, in an allowed orientation, inside its ULD and its cuts, outside the floor edge below δ, without overlapping other boxes, supported according to the setting and not on top of a fragile box, and the weight capacity of each ULD is respected. The validator also counts the ULDs whose CoG is out of the allowed deviation in the length or width direction (CoG_{xy}) and

above the maximum height (CoG_z, a constraint of Paquay et al. (2016) that Heßler et al. (2025) do not consider). All the 9000 solutions reported in this section are feasible. The comparison with Heßler et al. (2025) uses their instance-by-instance results (Tables 8, 10 and 12 of their online appendix). For each instance we count whether our solution is better (more boxes loaded or, with the same boxes, fewer ULDs), equal or worse, and we test the difference in the number of ULDs with the Wilcoxon signed-rank test. Each configuration is run five times; we report the mean and the standard deviation.

5.2 Results

Table 2 summarizes the results. In the Paquay setting, our algorithm uses 617.0 ULDs on average with $t = 0.1\,\mathrm{s}$ and 597.8 with $t = 1\,\mathrm{s}$, against 708 for the RGS of Heßler et al. (2025) and 1010 for the two-phase heuristic of Paquay et al. (2018a), a reduction of 12.9% and 15.6% with respect to the RGS. It is better in 91 and 111 instances, respectively, and worse in at most one. In the adapted setting, it uses 598.0 and 576.0 ULDs against 653 (8.4% and 11.8% fewer), and it is worse in at most one instance. In both settings the differences are highly significant (Wilcoxon signed-rank test, $p < 10^{-12}$). Moreover, almost no ULD violates the CoG constraints (at most two in 300 instances), whereas 11.2% (Paquay) and 7.2% (adapted) of the ULDs of the RGS have the CoG out of range.

In the setting with one ULD of each type, our algorithm uses 706.0 and 675.8 ULDs against 729 and leaves 22 and 21 boxes unloaded (out of 16 500) against 24, again with almost no CoG violation (3 ULDs) against 10.1% of the ULDs of the RGS. Instance by instance, it is better in 42 and 64 instances and worse in 22 and 20 ($p = 0.004$ and $p = 2 \cdot 10^{-8}$ for the number of ULDs). This setting is the closest one between the methods: with a limited number of ULDs, loading the boxes that only fit in a few ULD types matters more than the packing density, and the RGS gives the same importance to the weight balance and to the volume utilization ($I_w = 0.5$), which allows it to load boxes at the price of violating the CoG constraint (Section 5.6).

The results are almost deterministic: the five runs of each configuration differ by at most 0.5 ULDs on average, because in most ULDs the search ends before the time limit. Figure 1 shows the solution of one instance with one ULD of each type.

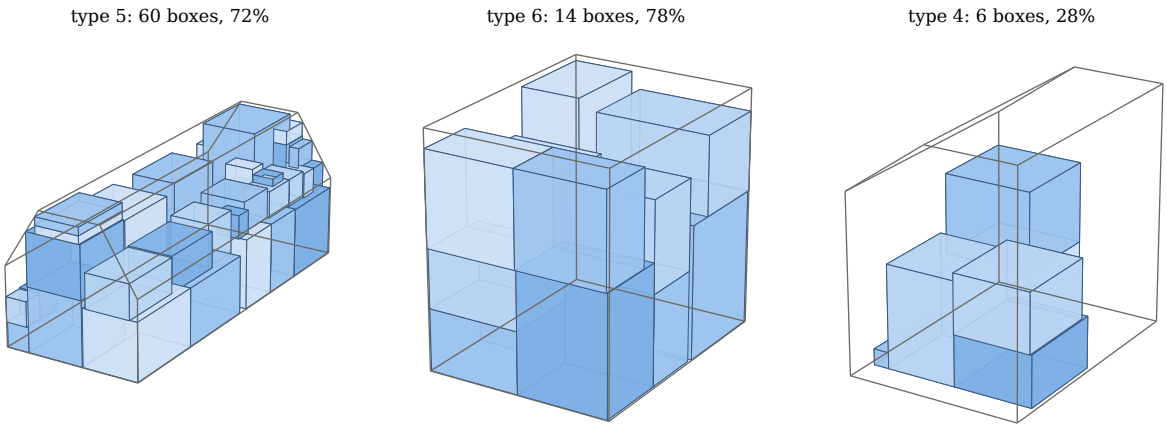


Figure 1: Solution of the instance with 80 boxes number 4 in the setting with one ULD of each type ($t = 0.1\,\mathrm{s}$): three ULDs (number of boxes and volume utilization). Type 5 has two bevelled top corners and type 4 one. The load of each ULD has been translated to center its CoG, which is within range in the three ULDs.

Table 2: Results on the 300 final instances (mean ± standard deviation over five runs). TPH: two-phase heuristic of Paquay et al. (2018a); RGS: randomized greedy search of Heßler et al. (2025). Better/worse: number of instances in which our solution is better/worse than the RGS. Vol.: total volume of the ULDs. CoG_{xy}, CoG_z: number (percentage) of ULDs with the CoG out of range. Time: mean time per instance (all ULDs).

Setting	Method	ULDs	Vol. (m^3)	Unloaded	Better/worse	CoG_{xy}	CoG_z	Time (s)
Paquay	TPH	1010	–	0		130 (12.9%)	–	≤ 12
	RGS	708	18 648	0		79 (11.2%)	–	≤ 0.5
	BSG, $t = 0.1\,\mathrm{s}$	617.0 ± 0.0	17 875	0	91.0 / 0	0 (0%)	0	0.62
	BSG, $t = 1\,\mathrm{s}$	597.8 ± 0.4	17 239	0	111.2 / 1	1 (0.2%)	0	5.4
Adapted	RGS	653	16 843	0		47 (7.2%)	–	≤ 1.3
	BSG, $t = 0.1\,\mathrm{s}$	598.0 ± 0.0	17 128	0	56.0 / 1	2 (0.3%)	0	1.2
	BSG, $t = 1\,\mathrm{s}$	576.0 ± 0.0	16 332	0	77.0 / 0	1 (0.2%)	0	10.3
One	RGS	729	16 166	24		74 (10.1%)	–	≤ 1.1
	BSG, $t = 0.1\,\mathrm{s}$	706.0 ± 0.0	16 050	22.0 ± 0.0	42.0 / 22.0	3 (0.4%)	0	1.6
	BSG, $t = 1\,\mathrm{s}$	675.8 ± 0.4	15 671	21.0 ± 0.0	64.4 / 20.2	2 (0.3%)	1	13.0

Volume and cost of the ULDs. Since the objective of Paquay et al. (2016) and Heßler et al. (2025) is the total volume of the ULDs rather than their number, Table 2 also reports the total volume of the ULDs (for the RGS, computed from the type of each ULD in Tables 9, 11 and 13 of their appendix). In the Paquay setting our solutions use 4.1% ($t = 0.1\,\mathrm{s}$) and 7.6% ($t = 1\,\mathrm{s}$) less ULD volume than the RGS, and with one ULD of each type 0.7% and 3.1% less, while loading more boxes. In the adapted setting with $t = 0.1\,\mathrm{s}$, however, they use 1.7% more volume than the RGS in spite of using 8.4% fewer ULDs, because they use more ULDs of the largest type; with $t = 1\,\mathrm{s}$ they use 3.0% less volume. The total cost of the ULDs given in the instance files behaves in the same way (3.9% and 7.2% less than the RGS in the Paquay setting, 2.1% more and 2.6% less in the adapted setting, 0.7% and 2.9% less with one ULD of each type). Our algorithm minimizes the number of ULDs; giving priority to the volume would require changing the selection of the next ULD type and the comparison of solutions in the elimination phase.

Results by instance size. Table 3 reports the results of the Paquay setting by instance size, in the format of Table 6 of Heßler et al. (2025): number of ULDs and median utilization of the ULDs of each group. Our algorithm uses the same number of ULDs as the RGS for $|I| = 10$, where every instance needs one ULD, and fewer ULDs in every other group; the difference grows with the number of boxes, from 3 ULDs (9%) for $|I| = 20$ to 20.6 ULDs (18%) for $|I| = 100$ with $t = 1\,\mathrm{s}$, and the median utilization for $|I| = 100$ increases from 53.0% to 70.4%. Table 4 reports the number of ULDs by instance size in the adapted settings. With one ULD of each type, the RGS leaves 23 of its 24 unloaded boxes in the instances with 100 boxes, where our algorithm leaves 16 ($t = 0.1\,\mathrm{s}$) and 13 ($t = 1\,\mathrm{s}$) with 1 and 14 fewer ULDs, whereas it leaves a few more boxes unloaded in the instances with 60–90 boxes (up to 4, against 1 for the RGS).

5.3 Comparison with a lower bound

A simple lower bound on the number of ULDs is the maximum of the total volume of the boxes divided by the volume of the largest ULD, the total weight divided by the largest weight capacity, and the size of a clique of pairwise incompatible boxes (two boxes are incompatible if they cannot be placed together in the bounding box of any ULD type). In these instances the bound is always given

Table 3: Results by instance size in the Paquay setting (30 instances per size): lower bound (LB, Section 5.3), number of ULDs ($|C|$), median utilization of the ULDs (u_{med}, %) and, for the RGS, number of ULDs with the CoG out of range (G_{vio}). The results of Paquay et al. (2018a) and the RGS are taken from Table 6 of Heßler et al. (2025); BSG: mean of five runs.

		Paquay et al. (2018a)		RGS			BSG, $t = 0.1$ s		BSG, $t = 1$ s	
$\|I\|$	LB	$\|C\|$	u_{med}	$\|C\|$	u_{med}	G_{vio}	$\|C\|$	u_{med}	$\|C\|$	u_{med}
10	30	47	33.2	30	34.3	0	30	34.3	30	34.3
20	30	54	34.6	35	42.1	3	32	46.3	32	47.0
30	30	72	36.6	47	44.7	6	41	52.4	35.2	57.5
40	30	83	37.9	60	46.5	4	53	53.4	48	59.6
50	39	90	42.2	66	47.5	5	59	54.8	58	56.8
60	52	108	41.5	80	49.6	10	63	57.1	64	52.8
70	58	126	45.7	82	54.0	6	77	58.7	75	56.8
80	60	129	47.0	93	54.5	12	79	61.9	76.2	63.6
90	60	141	47.5	102	53.5	13	88	62.1	87	64.3
100	63	160	50.2	113	53.0	20	95	67.1	92.4	70.4
Total	452	1010		708		79	617		597.8	

Table 4: Number of ULDs by instance size in the adapted settings (30 instances per size; in the setting with one ULD of each type, number of unloaded boxes in parentheses). BSG: mean of five runs.

	Adapted			One		
$\|I\|$	RGS	BSG, $t = 0.1$ s	BSG, $t = 1$ s	RGS	BSG, $t = 0.1$ s	BSG, $t = 1$ s
10	30	30	30	30 (0)	30 (0)	30 (0)
20	33	32	32	33 (0)	32 (0)	32 (0)
30	45	37	32	45 (0)	37 (0)	32 (0)
40	57	51	48	57 (0)	52 (0)	49 (0)
50	60	58	56	62 (0)	61 (0)	59 (0)
60	72	63	61	77 (0)	77 (0)	72.2 (1)
70	78	72	70	85 (0)	83 (0)	81 (0)
80	86	75	73	97 (0)	94 (2)	94 (3)
90	92	87	84	109 (1)	107 (4)	106.6 (4)
100	100	93	90	134 (23)	133 (16)	120 (13)
Total	653	598	576	729 (24)	706 (22)	675.8 (21)

by the volume. Our solutions reach the bound, and are therefore optimal, in 140 ($t = 0.1$ s) and 157 ($t = 1$ s) of the 300 instances of the Paquay setting, against 80 for the RGS. In the remaining instances our solutions use at most two ULDs more than the bound, but the bound is weak: it assumes that the boxes fill the largest ULD completely, whereas the non-final ULDs of our solutions are about 70% full. A stronger relaxation that also assigns the boxes to ULD types, with pairwise compatibility and area constraints, solved with the CP-SAT solver of OR-Tools, does not improve the bound in any instance: closing the remaining gaps requires proving that the boxes cannot be packed with such a density, a three-dimensional feasibility question that volume and pairwise arguments cannot answer. The 136 instances in which our best solution uses one ULD more than the bound remain open.

5.4 Running time

Heßler et al. (2025) solve each instance in 0.19 s on average (at most 0.5 s) in the Paquay setting, on an Apple M2 Pro. Figure 2 shows the number of ULDs obtained by our algorithm as a function of the mean time per instance, for time limits per ULD between 0.02 s and 2 s. With $t = 0.02$ s (0.16 s per instance on average, at most 0.52 s) our algorithm uses 638 ULDs, 9.9% fewer than the RGS, and it is worse in only one instance. Increasing the time limit keeps reducing the number of ULDs: 621 ULDs in 0.34 s, 617 in 0.62 s, 602 in 2.8 s and 596 in 10.5 s. Scaling the time of the RGS by the single-thread speed ratio of the machines (2.5), the RGS would need about 0.48 s per instance on our machine, between the mean times of our algorithm with $t = 0.05$ s (0.34 s, 621 ULDs) and $t = 0.1$ s (0.62 s, 617 ULDs): at a similar running time, our algorithm uses 12–13% fewer ULDs. This scaling is only indicative, since the two processors have different architectures.

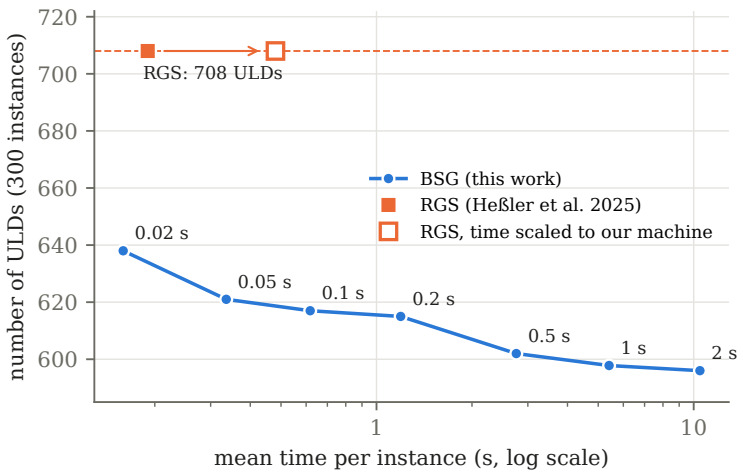


Figure 2: Number of ULDs in the 300 instances of the Paquay setting as a function of the mean time per instance (the labels are the time limits per ULD; mean of five runs for 0.1 s and 1 s, one run otherwise), and the RGS of Heßler et al. (2025): time on their machine (filled square) and scaled to ours by the single-thread speed ratio of 2.5 (empty square); the dashed line marks its 708 ULDs.

5.5 Contribution of the components

Table 5 reports the results when one component of the algorithm is removed ($t = 0.1$ s, one run). The three components that handle the CoG are essential for the weight distribution: without the

penalty in the value of the solutions, the number of ULDs decreases by 2–27 but 193–353 ULDs have the CoG out of range; without the translation of the load the CoG is out of range in 84–158 ULDs, and without the grouped reload in 20–31 ULDs, which also increases the number of ULDs (by 5–20) and, with one ULD of each type, the number of unloaded boxes (from 22 to 39). The penalty of the CoG in the evaluation of the blocks saves 7–9 ULDs without increasing the violations, the elimination of ULDs saves 1–13 ULDs, and the substructure saves 20–25 ULDs in the adapted settings. Removing all of them except the penalty and the translation increases the number of ULDs by 13 (Paquay), 36 (adapted) and 98 (one).

Table 5: Ablation ($t = 0.1\,\mathrm{s}$, one run): number of ULDs and number of ULDs with the CoG out of range in the length or width direction (in the setting *One*, also the number of unloaded boxes).

	Paquay		Adapted		One	
Without	ULDs	CoG_{xy}	ULDs	CoG_{xy}	ULDs (unloaded)	CoG_{xy}
– (complete algorithm)	617	0	598	2	706 (22)	3
CoG in the evaluation of the blocks	625	2	607	6	713 (29)	2
CoG penalty in the value	615	193	590	296	679 (17)	353
translation of the load	624	84	602	84	723 (28)	158
grouped reload	622	25	604	31	726 (39)	20
elimination of ULDs	620	0	599	2	719 (22)	4
substructure	–	–	618	1	731 (28)	6
all except penalty and translation	630	26	634	61	804 (39)	54

5.6 Number of ULDs versus weight distribution

The two methods treat the CoG differently: Heßler et al. (2025) combine the volume utilization and the weight balance in the score of a solution with weights $1 - I_w$ and I_w, with $I_w = 0.5$ in the adapted settings, whereas our penalty λ makes the CoG almost a hard constraint. As the ablation shows, relaxing it reduces the number of ULDs and of unloaded boxes at the price of many CoG violations. To compare the methods independently of this choice, Figure 3 shows the number of ULDs against the percentage of ULDs with the CoG out of range for $\lambda \in \{0, 0.5, 1, 2, 5, 12.66, 30\}$ ($t = 0.1\,\mathrm{s}$). In the Paquay and adapted settings the number of ULDs is almost insensitive to λ as long as $\lambda \geq 0.5$ (617–620 and 593–599 ULDs, with at most 9 ULDs out of range); only $\lambda = 0$, which ignores the CoG in the value of the solutions, saves a few more ULDs (616 and 591) with 31% and 49% of the ULDs out of range. In both settings every value $\lambda \geq 0.5$ dominates the RGS, which uses more ULDs and has more violations. In the setting with one ULD of each type, every value $\lambda \geq 5$, including the tuned one, dominates the RGS in the three criteria (705–710 ULDs, 22–23 unloaded boxes and 0.4% of the ULDs out of range, against 729, 24 and 10.1%); with $\lambda = 0.5$ the algorithm uses 699 ULDs and leaves 20 boxes unloaded with 2.7% of the ULDs out of range, and with $\lambda = 0$ it uses 679 ULDs and leaves 19 boxes unloaded, but half of the ULDs violate the CoG constraint.

6 Conclusions

We have adapted the block-based beam search BSG, originally designed for the single container loading problem, to the palletization of air cargo into ULDs with non-cuboid shapes, a floor edge and a substructure, weight capacities, fragile boxes, support constraints and CoG constraints. The ULD constraints are handled in the generation and placement of the blocks, the CoG constraint in

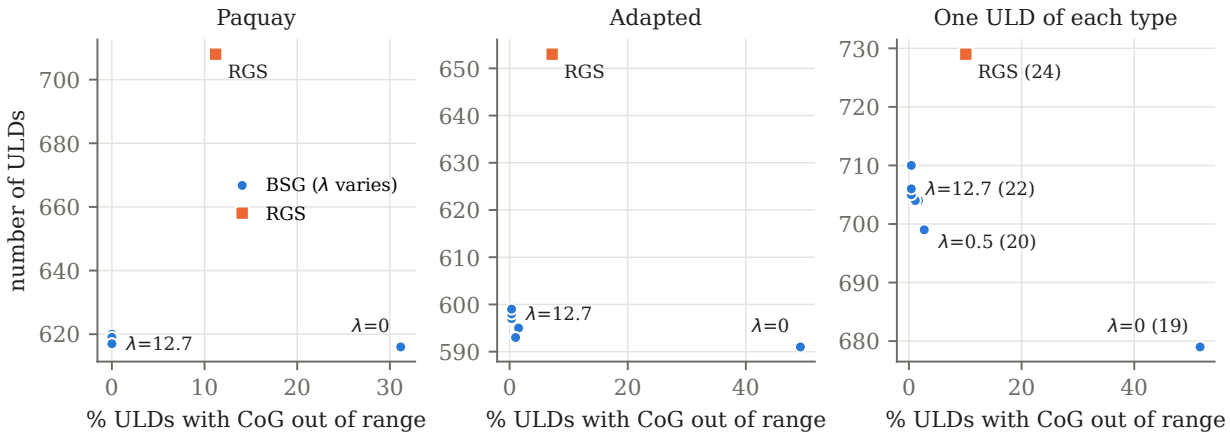


Figure 3: Number of ULDs against the percentage of ULDs with the CoG out of range in the length or width direction, for several values of the penalty λ ($t = 0.1$ s), and the RGS of Heßler et al. (2025). In the setting with one ULD of each type the number of unloaded boxes is given in parentheses.

the evaluation of the blocks, in the value of the solutions, with a final translation of the load, which is kept grouped by the anchoring of the spaces, and the ULDs are loaded sequentially and improved by eliminating ULDs and, when the number of ULDs is limited, by inserting the unloaded boxes with ruin and recreate.

On the 300 instances of Paquay et al. (2018a), and with the same parameters in all the settings, our algorithm uses 12.9–15.6% fewer ULDs than the RGS of Heßler et al. (2025) in the original setting and 8.4–11.8% fewer in the adapted one, and it is worse than the RGS in at most one instance, while almost none of its ULDs violates the CoG constraint. Its solutions reach a simple volume bound, and are therefore optimal, in up to 157 of the 300 original instances, against 80 for the RGS. With one ULD of each type, it loads more boxes with fewer ULDs and almost no CoG violation. With a time limit of only 0.02 s per ULD it already uses 9.9% fewer ULDs than the RGS and is worse in only one instance, and the trade-off between the number of ULDs and the CoG violations can be controlled with a single parameter.

These results show that the ideas that made block-based tree search successful in container loading, namely building the load with blocks of boxes, evaluating them with the VCS function and guiding a beam search with greedy completions, carry over to highly heterogeneous air cargo, provided that the blocks and their placement respect the support and fragility constraints and that the weight distribution is taken into account during the search.

Several directions remain open. A stronger lower bound would tell how far the remaining solutions are from the optimum. The sequential framework could be replaced by a search that decides the assignment of the boxes to the ULDs jointly, which matters most when the number of ULDs is limited. Other practical constraints, such as load bearing capacities, the grouping of the boxes by destination or the balance of the whole aircraft, could also be integrated into the evaluation of the blocks and of the solutions.

References

Ramón Alvarez-Valdes, Francisco Parreño, and José Manuel Tamarit. A GRASP/Path Relinking algorithm for two- and three-dimensional multiple bin-size bin packing problems. *Computers &*

Operations Research, 40(12):3081–3090, 2013.

Ignacio Araya and María-Cristina Riff. A beam search approach to the container loading problem. *Computers & Operations Research*, 43:100–107, 2014.

Ignacio Araya, Keytel Guerrero, and Eduardo Nuñez. VCS: A new heuristic function for selecting boxes in the single container loading problem. *Computers & Operations Research*, 82:27–35, 2017.

Ignacio Araya, Mauricio Moyano, and Cristobal Sanchez. A beam search algorithm for the biobjective container loading problem. *European Journal of Operational Research*, 286(2):417–431, 2020.

Mauro Maria Baldi, Guido Perboli, and Roberto Tadei. The three-dimensional knapsack problem with balancing constraints. *Applied Mathematics and Computation*, 218(19):9802–9818, 2012.

Eberhard E. Bischoff and M. S. W. Ratcliff. Issues in the development of approaches to container loading. *Omega*, 23(4):377–390, 1995.

Andreas Bortfeldt and Gerhard Wäscher. Constraints in container loading – a state-of-the-art review. *European Journal of Operational Research*, 229(1):1–20, 2013.

Felix Brandt and Stefan Nickel. The air cargo load planning problem – a consolidated problem definition and literature review on related problems. *European Journal of Operational Research*, 275(2):399–410, 2019.

Felix T. S. Chan, Rajat Bhagwat, Nishant Kumar, Manoj K. Tiwari, and P. Lam. Development of a decision support system for air-cargo pallets loading problem: A case study. *Expert Systems with Applications*, 31(3):472–485, 2006.

Teodor Gabriel Crainic, Guido Perboli, and Roberto Tadei. Extreme point-based heuristics for three-dimensional bin packing. *INFORMS Journal on Computing*, 20(3):368–384, 2008.

A. Paul Davies and Eberhard E. Bischoff. Weight distribution considerations in container loading. *European Journal of Operational Research*, 114(3):509–527, 1999.

Tobias Fanslau and Andreas Bortfeldt. A tree search algorithm for solving the container loading problem. *INFORMS Journal on Computing*, 22(2):222–235, 2010.

Oluf Faroe, David Pisinger, and Martin Zachariasen. Guided local search for the three-dimensional bin-packing problem. *INFORMS Journal on Computing*, 15(3):267–283, 2003.

Mikele Gajda, Alessio Trivella, Renata Mansini, and David Pisinger. An optimization approach for a complex real-life container loading problem. *Omega*, 107:102559, 2022.

Lucas Guesser, Thiago Alves de Queiroz, and Flávio Keidi Miyazawa. A tree search-based heuristic for the three-dimensional single container loading problem. *European Journal of Operational Research*, 333:337–348, 2026.

Katrin Heßler, Timo Hintsch, and Lukas Wienkamp. A fast optimization approach for a complex real-life 3D Multiple Bin Size Bin Packing Problem. *European Journal of Operational Research*, 2025. arXiv:2410.01445.

Manuel López-Ibáñez, Jérémie Dubois-Lacoste, Leslie Pérez Cáceres, Mauro Birattari, and Thomas Stützle. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016.

Virginie Lurkin and Michael Schyns. The airline container loading problem with pickup and delivery. *European Journal of Operational Research*, 244(3):955–965, 2015.

Silvano Martello, David Pisinger, and Daniele Vigo. The three-dimensional bin packing problem. *Operations Research*, 48(2):256–267, 2000.

A. C. P. Mesquita and C. A. A. Sanches. Air cargo load and route planning in pickup and delivery operations. arXiv:2606.26404, 2026.

Céline Paquay, Michael Schyns, and Sabine Limbourg. A mixed integer programming formulation for the three-dimensional bin packing problem deriving from an air cargo application. *International Transactions in Operational Research*, 23(1-2):187–213, 2016.

Céline Paquay, Sabine Limbourg, and Michael Schyns. A tailored two-phase constructive heuristic for the three-dimensional Multiple Bin Size Bin Packing Problem with transportation constraints. *European Journal of Operational Research*, 267(1):52–64, 2018a.

Céline Paquay, Sabine Limbourg, Michael Schyns, and José Fernando Oliveira. MIP-based constructive heuristics for the three-dimensional Bin Packing Problem with transportation constraints. *International Journal of Production Research*, 56(4):1581–1592, 2018b.

Francisco Parreño, Ramón Alvarez-Valdes, José Manuel Tamarit, and José Fernando Oliveira. A maximal-space algorithm for the container loading problem. *INFORMS Journal on Computing*, 20(3):412–422, 2008.

Francisco Parreño, Ramón Alvarez-Valdes, José Fernando Oliveira, and José Manuel Tamarit. A hybrid GRASP/VND algorithm for two- and three-dimensional bin packing. *Annals of Operations Research*, 179(1):203–220, 2010.

PassMark Software. CPU Benchmarks: single thread performance. https://www.cpubenchmark.net. Accessed October 2026.

Gerhard Schrimpf, Johannes Schneider, Hermann Stamm-Wilbrandt, and Gunter Dueck. Record breaking optimization results using the ruin and recreate principle. *Journal of Computational Physics*, 159(2):139–171, 2000.

Alessio Trivella and David Pisinger. The load-balanced multi-dimensional bin-packing problem. *Computers & Operations Research*, 74:152–164, 2016.

Gerhard Wäscher, Heike Haußner, and Holger Schumann. An improved typology of cutting and packing problems. *European Journal of Operational Research*, 183(3):1109–1130, 2007.

Xiaozhou Zhao, Julia A. Bennell, Tolga Bektaş, and Kath Dowsland. A comparative review of 3D container loading algorithms. *International Transactions in Operational Research*, 23(1-2): 287–320, 2016.

Wenbin Zhu and Andrew Lim. A new iterative-doubling Greedy–Lookahead algorithm for the single container loading problem. *European Journal of Operational Research*, 222(3):408–417, 2012.